# ParkWise — Automated Parking Lot & Conveyor Control System

# Technical Architecture & Engineering Case Study

## 1. Executive Summary & System Architecture

### Problem Statement & Operational Context

Accelerating urban growth and rising vehicle density in modern metropolitan centers have pushed conventional multi-story parking facilities to their structural and operational limits. Traditional facilities rely heavily on manual driver navigation and static spot allocation, introducing severe operational friction. Common failure modes include throughput bottlenecks at entry and exit gates, inefficient space utilization, elevated carbon emissions from vehicles idling while searching for spaces, and an increased rate of minor collisions within tight parking structures.

While automated mechanical conveyor systems offer a high-density alternative, they introduce critical single-point-of-failure risks. Mechanical conveyor jams, motor thermal overloads, optical sensor misalignments, and electronic controller faults can instantly freeze entire facility floors, trapping parked vehicles and halting incoming traffic. ParkWise was engineered as an enterprise-grade control software platform to eliminate these systemic operational risks. By integrating automated hardware orchestration, real-time concurrent diagnostic state monitoring, dynamic 3D spatial allocation heuristics, and fault-tolerant financial settlement processing, ParkWise provides a resilient, highly available solution for modern automated parking infrastructure.

### Architectural Standard: Entity-Control-Boundary (ECB) Pattern

To maintain strict maintainability, scalability, and system isolation, the core architecture of ParkWise enforces the Entity-Control-Boundary (ECB) structural pattern. This design decouples presentation layers from core business rules and underlying data storage engines, preventing tight coupling across the system lifecycle.

- **Boundary Layer (Presentation & Interfaces)**: Formulates presentation components without embedding core domain logic. It encapsulates administrative desktop client management views, engineering telemetry dashboards, and subscriber self-service touch-screen terminal interfaces.
- **Control Layer (Transactional & Operational Core)**: Serves as the transactional engine of the application. Controllers ingest real-time hardware telemetry, execute spatial positioning heuristics, manage multithreaded diagnostic monitoring loops, calculate dynamic tariff structures, and govern database transaction boundaries.
- **Entity Layer (Domain Model & Business Rules)**: Encapsulates domain model data objects and business invariants (e.g., `ParkingSession`, `Conveyor`, `Vehicle`, and `PriceRate`), ensuring persistence changes or driver updates do not pollute core business logic.

This structural separation ensures that hardware communication protocols or database drivers (e.g., transitioning from JDBC/UCanAccess to distributed enterprise database clusters) can be modified independently without impacting user interface components or core spatial allocation algorithms.

## Role-Based Access Control (RBAC) & Portal Architecture

ParkWise enforces strict operational security through Role-Based Access Control (RBAC), dividing application access into two specialized, purpose-built portals:

- **Administrative Operations Portal**: Tailored for facility engineers, maintenance specialists, and system managers. Features real-time floor telemetry views, live conveyor state displays, manual conveyor interlock controls, dynamic tariff JSON ingestion tools, and executive business intelligence report generators.
- **Subscriber Self-Service Portal**: Optimized for customer-facing touch-screen terminals and mobile interfaces. Handles automated vehicle intake registration, real-time parking duration fee estimations, secure digital payment processing, and automatic barrier gate release triggering.

## Tech Stack Breakdown

| Layer / Subsystem | Technology | Version | Engineering Role & Operational Purpose |
|---|---|---|---|
| Runtime Environment | Java SE | JDK 17 LTS | Core execution environment utilizing LTS performance enhancements and thread pools. |
| UI Framework | Java Swing | Native JDK | Cross-platform desktop framework delivering responsive UI components for admin views. |
| Persistence Bridge | JDBC / UCanAccess | 5.0.0 | Pure Java JDBC driver bridging transactional relational queries with backend Access engines. |
| Database Engine | Microsoft Access | MS JET/ACE | Embedded relational database handling normalized schemas, foreign keys, and indexes. |
| Reporting Engine | JasperReports Library | 6.20.0 | Enterprise compilation engine converting dynamic JRXML templates into |

| Layer / Subsystem | Technology | Version | Engineering Role & Operational Purpose |
|---|---|---|---|
| | | | formal PDF reports. |
| Data Interchange | org.json.simple | 1.1.1 | Lightweight JSON parser used for annual tariff schedule ingestion and configuration loading. |

## 2. Core Engineering Algorithms & Concurrency

### 3D Spatial Optimization Engine

ParkWise features a multi-constraint heuristic spatial allocation engine designed to minimize conveyor travel distance, lower mechanical stress on drive assemblies, and maximize structural space utilization across multi-level facilities.

#### Dimensional & Structural Constraints

When a vehicle arrives at the intake bay, the positioning controller evaluates candidate spaces against mandatory physical constraints before distance scoring:

1. **Vehicle Size Classification**: Vehicles are classified into Small (S), Medium (M), and Large (L) dimensions. A candidate parking space `S_slot` is valid for vehicle `V` only if `Size(V) <= Capacity(S_slot)`.
2. **Conveyor Weight Thresholds**: Each floor conveyor `C` has a maximum rated load `W_max`. Allocation strictly enforces `W_vehicle <= W_conveyor_max` to prevent mechanical strain or structural interlock trips.

#### 3D Spatial Distance Scoring Formula

Vertical floor transit via elevator lifts consumes significantly more power and time than horizontal transit on a single floor. To model this operational cost, the allocation engine applies an asymmetric floor penalty score when evaluating candidate spaces from

conveyor origin $(x_c, y_c, Floor_{conveyor})$ to space destination $(x_s, y_s, Floor_{space})$:

**Score = (|Floor_conveyor - Floor_space| × 1,000,000) + √((x_c - x_s)² + (y_c - y_s)²)**

Multiplying the vertical floor differential by a heavy multiplier (1,000,000) guarantees that the algorithm exhausts all suitable vacant spaces on the conveyor's current floor before initiating inter-floor elevator transfers. This reduces elevator cycles and maximizes throughput during peak arrival windows.

### Spatial Allocation Decision Pipeline

| Step | Pipeline Stage | Operational Action & Logic |
|---|---|---|
| 1 | Intake & Physical Validation | Ingest vehicle parameters (weight, dimensions) and verify against intake sensor bounds. |
| 2 | Vacancy & Size Filtering | Filter active parking spaces for current availability and dimensional capacity compatibility. |
| 3 | Heuristic Score Computation | Apply 3D scoring formula with vertical floor penalty to all candidate vacant spaces. |
| 4 | Target Assignment | Select space with the minimum numerical score and dispatch transport conveyor commands. |

## Concurrent Diagnostic State Machine

Conveyor mechanical reliability is managed through a Finite State Machine (FSM) linked to parallel hardware diagnostic execution loops.

### State Machine Definition

The system governs conveyor operations across five strictly defined operational states:

- `OffStable`: Power completely removed or hardware isolated for scheduled maintenance.
- `Initializing`: System bootstrap phase performing memory allocation and initial sensor calibrations.
- `Operational_Idle`: Hardware online, fully calibrated, and awaiting positioning commands.
- `Operational_Executing`: Active vehicle placement or retrieval run in progress.
- `Paused`: Safe state triggered by diagnostic anomalies, safety interlocks, or manual admin overrides.

### Multithreaded Diagnostic Execution Architecture

Diagnostic sweeps execute asynchronously using an `ExecutorService` thread pool with two dedicated asynchronous threads:

1. **Mechanical Diagnostic Thread**: Monitors belt tension sensors, optical beam blockage indicators, roller friction metrics, and drive motor torque draw.
2. **Electronic Diagnostic Thread**: Validates proximity sensor alignment, relay control continuity, gate actuator states, and controller signal integrity.

### Diagnostic & Fault Handling Flow

| Trigger Condition | Diagnostic Evaluation | Outcome & Action | Resulting State |
|---|---|---|---|
| Periodic Sweep or Manual Trigger | Mechanical & Electronic threads run parallel checks (10-min timeout). | All integrity checks pass successfully within timeout window. | Operational_Idle |

| Trigger Condition | Diagnostic Evaluation | Outcome & Action | Resulting State |
|---|---|---|---|
| Sensor Anomaly or Timeout (<= 3 Retries) | Controller detects fault or response lag during execution sweep. | Increment retry counter and trigger re-diagnostic with backoff. | Re-Trigger Active Check |
| Persistent Fault (> 3 Retries) | 3 consecutive diagnostic retries fail or time out. | Lock floor from new allocations, initiate safety halt, and alert Admin Portal. | Paused |

**Fault Resilience & Retry Policy**

- Diagnostic sweeps operate under a strict **10-minute (600,000 ms) timeout** per execution cycle.
- If an anomaly is detected or a response times out, the controller triggers up to **3 automatic retry attempts** using exponential backoff delay intervals.
- If all 3 retries fail, the conveyor transitions immediately to the `Paused` state, isolates the affected floor from new vehicle allocations, and dispatches an emergency notification to the Administrative Operations Portal.

# 3. Data Persistence & Financial Logic

## Relational Schema & ACID Guarantees

ParkWise manages transactional data persistence using a third-normal-form (3NF) relational schema, enforcing strict referential integrity across core entity domains: `Client`, `Vehicle`, `ParkingLot`, `ParkingSpace`, `Conveyor`, `ParkingSession`, `ClubMembership`, `PreferredLotSelection`, and `PriceRate`.

To ensure ACID compliance during high-throughput operational windows (e.g., peak arrival or departure spikes), space allocation and session initialization execute inside isolated database transaction blocks. If a conveyor assignment or space reservation fails at any point during intake, the entire session transaction rolls back, preserving accurate system vacancy counts and preventing ghost allocations.

## Automated Schema Migrations

Schema evolution is managed automatically at runtime via a specialized migration engine (`DbMigrations`). On application startup, the engine verifies database metadata against defined entity rules, inspects index constraints, and reconciles state inconsistencies (such as aligning space availability flags in `ParkingSpace` with open records in `ParkingSession`). Necessary DDL schema alterations are applied automatically without requiring manual database administrator intervention.

## Dynamic Pricing & Tariff Ingestion

The tariff engine ingests annual financial configurations dynamically from structured JSON definitions, allowing operators to update pricing schedules across facilities seamlessly:

```
{
  "facilityId": "LOT-NORTH-01",
  "effectiveYear": 2026,
  "currency": "USD",
  "baseHourlyRate": 4.50,
  "maxDailyRate": 35.00,
  "clubDiscountPercentage": 15.0
}
```

### Stepped Duration Billing Algorithm

Parking duration is evaluated using entry and departure timestamps, applying ceiling rounding logic to calculate final billable units:

**Billed Hours = Ceiling(Duration in Minutes / 60.0)**

**Gross Fee = Min(Billed Hours × BaseHourlyRate, MaxDailyRate)**

## Customer Club Loyalty Rules

The billing engine integrates multi-tiered customer loyalty logic during session finalization:

- **Tiered Member Discounts**: Valid club members receive immediate percentage discounts applied directly to calculated gross parking fees.

- **Preferred Facility Mapping**: Members designate a primary home facility (`PreferredLotSelection`).
- **Monthly Preferred Waiver**: The system inspects session history for the current calendar month. The first completed session each month at the member's preferred facility is waived entirely (Net Fee = $0.00).

# 4. Business Intelligence & Enterprise Reporting

## JasperReports Integration & PDF Generation Pipeline

ParkWise embeds the JasperReports engine to compile and export formal financial summaries, operational logs, and executive PDFs directly from underlying relational data.

### Report Compilation Stage

| Stage | Pipeline Phase | System Operation |
|---|---|---|
| 1 | Template Compilation | Compile dynamic JRXML template definitions into JasperReport object binaries. |
| 2 | Database Population | Execute relational SQL queries to feed live database datasets into the Report Fill Engine. |
| 3 | Document Export | Render compiled report objects into formatted PDF documents for executive distribution. |

### Multi-Level Grouping & Sorting Pipeline

The Annual Parking Lot Summary Report aggregates performance metrics across multiple facilities. Query pipelines group records by facility (`ParkingLot`) and sub-group them by vehicle color classifications, applying strict ordering rules:

**ORDER BY City DESC, TotalRevenue ASC**

This structured grouping gives operations managers immediate visibility into geographic revenue trends and facility utilization rates.

## Customer Club Growth Analytics

The analytics engine incorporates a time-series aggregation model to track loyalty program expansion over time:

1. **Baseline Membership Tracking**: Establishes active membership counts at the start of the evaluation period.
2. **Monthly Registration Ingestion**: Groups new `ClubMembership` records by registration timestamps across calendar months.
3. **Churn & Expiration Adjustments**: Deducts expired or cancelled memberships to maintain precise active counts.
4. **Cumulative Trajectory Output**: Derives net cumulative membership growth trajectories for executive dashboards.

## Key Engineering Takeaways

- **Architectural Isolation**: Enforcing the ECB pattern cleanly separates user interface components, transactional logic, and persistence layers.
- **High-Availability Diagnostic Loop**: Multithreaded state monitoring prevents localized hardware failures from creating facility-wide outages.
- **3D Algorithmic Efficiency**: Asymmetric floor scoring minimizes energy-intensive vertical elevator transit while optimizing space usage.
- **Extensible Enterprise Reporting**: Dynamic JSON pricing ingestion and JasperReports integration deliver robust financial settlement and business intelligence.